Praktikum JavaScript & TypeScript

Modul 3 - Array dan Method Array

1. Tujuan Praktikum

Setelah mengikuti praktikum ini, peserta mampu:

- Menyimpan data berbentuk array of object
- Memakai `forEach`, `map`, `filter`, `find`, `some`, `every`, `includes`, dan `reduce`
- Merangkai beberapa method menjadi satu alur (*chaining*)
- Membedakan method yang mengubah array asli dan yang tidak
- Menjelaskan kapan memakai `for` dan kapan memakai `forEach`

2. Dasar Teori

2.1. Array of Object

Data nyata jarang berupa angka saja. Satu produk punya nama, harga, dan stok. Data seperti ini disimpan dalam *object*, lalu banyak object disimpan dalam array. Object dibahas lebih dalam di Modul 4. Di modul ini cukup tahu cara membuat dan membaca propertinya.

```
const pasien = [
  { nama: "Budi", umur: 40, poli: "Umum" },
  { nama: "Sari", umur: 7, poli: "Anak" },
  { nama: "Joko", umur: 65, poli: "Umum" },
];
console.log(pasien[1].nama);   // "Sari"
console.log(pasien.length);    // 3
```

Listing 1: array of object

2.2. Method Array dan Callback

Setiap method di modul ini menerima *callback*: fungsi yang dipanggil sekali untuk **setiap** elemen. Callback menerima elemen saat ini, lalu (opsional) index-nya. Ingat dari Modul 2: kita mengirim fungsinya, bukan hasil pemanggilannya.

Method	Mengembalikan	Ubah asli?
`forEach`	`undefined` (hanya menjalankan aksi)	Tidak
`map`	array baru, panjang sama	Tidak
`filter`	array baru, elemen yang lolos	Tidak
`find`	elemen pertama yang cocok, atau `undefined`	Tidak
`findIndex`	index pertama yang cocok, atau `-1`	Tidak
`some`	`true` jika minimal satu cocok	Tidak
`every`	`true` jika semua cocok	Tidak
`includes`	`true` jika nilai ada di array	Tidak
`reduce`	satu nilai hasil akumulasi	Tidak
`sort`	array yang **sama**, sudah diurutkan	**Ya**

`push`	panjang baru	**Ya**

2.2.1. forEach

`forEach` menjalankan aksi untuk setiap elemen. Hasilnya selalu `undefined`, jadi `forEach` dipakai untuk efek samping seperti mencetak.

```
pasien.forEach((p, i) => {
  console.log(`${i + 1}. ${p.nama}`);
});
```

Listing 2: forEach

2.2.2. map: ubah setiap elemen

```
const namaSaja = pasien.map((p) => p.nama);
// ["Budi", "Sari", "Joko"]
const hargaPPN = [10000, 20000].map((h) => h + h * 11 / 100);
// [11100, 22200]
```

Listing 3: map

2.2.3. filter: saring elemen

```
const pasienUmum = pasien.filter((p) => p.poli === "Umum");
// 2 object: Budi dan Joko
const kosong = pasien.filter((p) => p.umur > 100);
// [] (array kosong, bukan null)
```

Listing 4: filter

2.2.4. find dan findIndex: cari satu elemen

`find` berhenti begitu menemukan elemen pertama yang cocok. Jika tidak ada yang cocok, hasilnya `undefined`, jadi selalu cek hasilnya sebelum membaca properti.

```
const anak = pasien.find((p) => p.poli === "Anak");
console.log(anak.nama);        // "Sari"

const gigi = pasien.find((p) => p.poli === "Gigi");
console.log(gigi);             // undefined
console.log(gigi.nama);        // TypeError: Cannot read properties of undefined
```

Listing 5: find

2.2.5. some, every, includes

```
pasien.some((p) => p.umur >= 60);   // true  (Joko)
pasien.every((p) => p.umur >= 18);  // false (Sari 7 tahun)

const poliBuka = ["Umum", "Anak", "Gigi"];
poliBuka.includes("Anak");          // true
poliBuka.includes("Mata");          // false
```

Listing 6: some, every, includes

⚠ Perhatian

Array JavaScript **tidak** punya method `contains()`. Memanggilnya menghasilkan `TypeError: poliBuka.contains is not a function`. Nama `contains` berasal dari Java dan C#. Di JavaScript, `contains()` hanya ada di DOM, misalnya `element.classList.contains("aktif")`. Untuk array dan string, pakai `includes()`.

⚠ Perhatian

`includes` membandingkan dengan `===`. Untuk object, `===` membandingkan **alamat** object, bukan isinya. Jadi `pasien.includes({ nama: "Budi", umur: 40, poli: "Umum" })` menghasilkan `false`, karena object baru punya alamat berbeda. Untuk mencari object berdasarkan isinya, pakai `some` atau `find`.

2.2.6. reduce: gabungkan menjadi satu nilai

`reduce(callback, nilaiAwal)` membawa sebuah *akumulator* dari elemen ke elemen. Callback menerima `(akumulator, elemen)` dan mengembalikan akumulator baru.

```
const tagihan = [150000, 75000, 300000];
const total = tagihan.reduce((acc, t) => acc + t, 0);
// putaran: 0+150000 -> 150000+75000 -> 225000+300000 = 525000

const totalUmur = pasien.reduce((acc, p) => acc + p.umur, 0); // 112
```

Listing 7: reduce

⚠ Perhatian

Selalu tulis nilai awal (argumen kedua). Tanpa nilai awal, `reduce` memakai elemen pertama sebagai akumulator. Untuk array angka ini kebetulan benar. Untuk array object, akumulator pertama adalah **object**, sehingga `acc + p.umur` menyambung string `"[object Object]..."`. Untuk array kosong tanpa nilai awal, `reduce` melempar `TypeError`.

2.2.7. sort

`sort` mengurutkan array **asli** dan mengembalikan array yang sama. Tanpa callback, `sort` mengurutkan sebagai **string**.

```
[10, 9, 1].sort();                 // [1, 10, 9]  <- urut sebagai teks
[10, 9, 1].sort((a, b) => a - b);  // [1, 9, 10]  naik
[10, 9, 1].sort((a, b) => b - a);  // [10, 9, 1]  turun

const urutUmur = [...pasien].sort((a, b) => a.umur - b.umur);
// pasien asli tidak berubah, karena yang diurutkan salinannya
```

Listing 8: sort

Callback `sort` mengembalikan angka negatif jika `a` harus di depan `b`, positif jika sebaliknya, dan `0` jika sama. `[...pasien]` membuat salinan array (*spread*, dibahas di Modul 4). Node 20 ke atas juga punya `toSorted()` yang mengembalikan array baru.

⚠ Perhatian

Mengubah array asli bisa merusak bagian program lain yang memakai array yang sama. Bug seperti ini sulit dilacak karena error tidak muncul di tempat perubahan terjadi. Biasakan menyalin array sebelum `sort`.

2.3. Chaining

Karena `map` dan `filter` mengembalikan array, hasilnya bisa langsung diproses method berikutnya.

```
const tagihanPasien = [
  { nama: "Budi", poli: "Umum", biaya: 150000 },
  { nama: "Sari", poli: "Anak", biaya: 200000 },
  { nama: "Joko", poli: "Umum", biaya: 90000 },
];
const totalUmum = tagihanPasien
  .filter((t) => t.poli === "Umum")
  .map((t) => t.biaya)
  .reduce((acc, b) => acc + b, 0);
// 240000
```

Listing 9: chaining: total tagihan pasien umum

Baca rantai seperti kalimat: saring pasien umum, ambil biayanya, lalu jumlahkan.

2.4. for vs forEach

`for` / `for...of`	`forEach`
Bisa berhenti dengan `break` atau `return` dari fungsi	Tidak bisa berhenti. `return` di callback hanya melompati elemen itu
Cocok untuk mencari, atau berhenti saat kondisi terpenuhi	Cocok untuk aksi pada **semua** elemen
Bisa memakai `await` dengan benar (Modul 7)	Tidak menunggu `await` di dalam callback

```
[1, 2, 3].forEach((n) => {
  if (n === 2) return;   // hanya melompati n = 2
  console.log(n);
});
// 1
// 3
```

Listing 10: return di forEach tidak menghentikan perulangan

✎ Catatan

Saat ditanya “kenapa pakai `filter` dan `map`, bukan `for`?”, jawaban yang baik: method array menunjukkan **niat** (menyaring, mengubah, menjumlahkan) dan tidak mengubah data asli. `for` tetap tepat ketika kamu perlu berhenti lebih awal atau logikanya terlalu rumit untuk satu callback.

Coba Sendiri

Buat file `coba3.js` berisi array `pasien` dari Listing 1. Tulis tebakanmu di laporan sebelum menjalankan.

```
const angka = [5, 25, 100, 1];
console.log(angka.sort());
console.log(angka);
console.log(pasien.map((p) => p.umur > 18));
console.log(pasien.filter((p) => p.umur > 18).length);
console.log(pasien.find((p) => p.umur > 18).nama);
console.log([].reduce((a, b) => a + b, 0));
const hasil = pasien.forEach((p) => p.nama);
console.log(hasil);
```

1. Tebakan mana yang salah? Jelaskan penyebabnya.
2. Apa beda hasil `map` dan `filter` pada baris ketiga dan keempat?
3. Hapus `, 0` pada baris `reduce`, lalu jalankan. Catat error-nya dan jelaskan.
4. Dengan chaining, tampilkan nama pasien poli Umum dalam satu string dipisah koma: `Budi, Joko`. Petunjuk: method `join(", ")`.

Sertakan screenshot hasil tiap langkah.